



Assignment 1 - Mini Arithmetic Logic Unit

Student Info:

Full Name: Shipra Singh

Student ID: 25370745

1. Introduction

For this assignment I designed and tested a 6-bit Mini ALU in Verilog and loaded it onto the Basys-3 FPGA board. The ALU takes two signed 6-bit inputs A and B in two's complement, along with a 3-bit control signal fxn, and outputs a 6-bit result X. One of the main requirements was to build the design by reusing code from previous labs rather than starting from scratch. The target device was the Xilinx Artix-7 on the Basys-3 board. Table 1 below summarises the eight operations the ALU must perform.

fxn	Operation	Output X
000	Pass A	$X = A$
001	Pass B	$X = B$
010	Negate A	$X = -A$
011	Negate B	$X = -B$
100	Signed Less-Than	$X = A < B$ (0 or 1)
101	Bitwise XNOR	$X = A \text{ XNOR } B$
110	Addition	$X = A + B$
111	Subtraction	$X = A - B$

Table 1: Mini-ALU function select table

2. Design Strategy

My approach was to break the ALU down into small, clearly defined sub-modules that each do one job and can be tested in isolation. The top-level module `minimum_alu` only contains instantiations and no logic of its own, which was a specific requirement of the assignment. I identified which modules from previous labs I could reuse directly and which new ones I needed to write.

2.1 Modules Reused Without Modification

The following modules from earlier labs were used exactly as written, with no changes made to the code:

- **two_bit.v:** Contains the `gt2` (2-bit greater-than) and `eq2` (2-bit equality comparator) modules I wrote in Lab A at the gate level. Brought in unchanged as building blocks for the comparator chain.
- **eight_bit.v:** The `gte8` (8-bit greater-than-or-equal) comparator from Lab B, built from four `gt2` and four `eq2` instances. Importantly I kept this at its full 8-bit width rather than cutting it down to 6 bits, as instructed.



- **full_adder.v:** The single-bit FullAdder from Lab C. Used unchanged as the basic cell inside the ripple adder.
- **6bit_ripple_adder.v:** The ripple_adder_6bit adder/subtractor from Lab C and Lab E. When sel=0 it adds, when sel=1 it subtracts using two's complement inversion. No modifications needed.

2.2 Modules Adapted from Previous Labs

One module required adaptation rather than direct reuse or fresh writing:

- **less_than_six_bit.v:** This module reuses gt_8bit (eight_bit.v) as a sub-component but wraps it with new logic to make it work for signed 6-bit comparison. The gt_8bit module itself was not touched, but I adapted the way it is used: I flip the MSB of each 6-bit input before zero-extending to 8 bits and passing them in. This converts two's complement to offset binary, where the unsigned ordering matches the signed ordering, so the unmodified gt_8bit gives the correct signed result. The adaptation is entirely in the new wrapper logic, not inside the reused module.

2.3 New Modules Written for This Assignment

I wrote five new modules specifically for this assignment:

- **adder_input.v:** Controls what gets fed into the ripple adder depending on fxn. For negation I set one input to zero so the adder computes 0 minus the operand. For subtraction I pass A and B directly with sel=1.
- **less_than_six_bit.v:** A signed 6-bit less-than comparator. I reused gt_8bit without touching it by flipping the MSB of each input before passing them in. This converts two's complement to offset binary, which lets the unsigned gt_8bit make a correct signed comparison.
- **six_bit_xnor.v:** A simple bitwise XNOR across all 6 bits, implemented as: assign result = ~(A ^ B).
- **multiplexer_output.v:** An 8-to-1 mux that picks the right computed result to send to output X based on fxn.
- **minimum_alu.v:** The top-level module. Contains only the five instantiations wired together, with no logic.

2.4 Module Hierarchy

The full hierarchy of the design, as shown in the Vivado Sources panel in Section 6, is:

```
├── minimum_alu                (minimum_alu.v)      [TOP]
├── adder_input                (adder_input.v)
├── ripple_adder_6bit          (6bit_ripple_adder.v)
│   ├── FullAdder x6          (full_adder.v)
├── less_than_six_bit          (less_than_six_bit.v)
│   ├── gt_8BIT               (eight_bit.v)
│   │   ├── gt2 x4            (two_bit.v)
│   │   └── eq2 x4            (two_bit.v)
├── six_bit_xnor               (six_bit_xnor.v)
└── multiplexer_output         (multiplexer_output.v)
```



Figure A below shows a block diagram of the top-level design, illustrating how the sub-modules connect together and what signals pass between them.

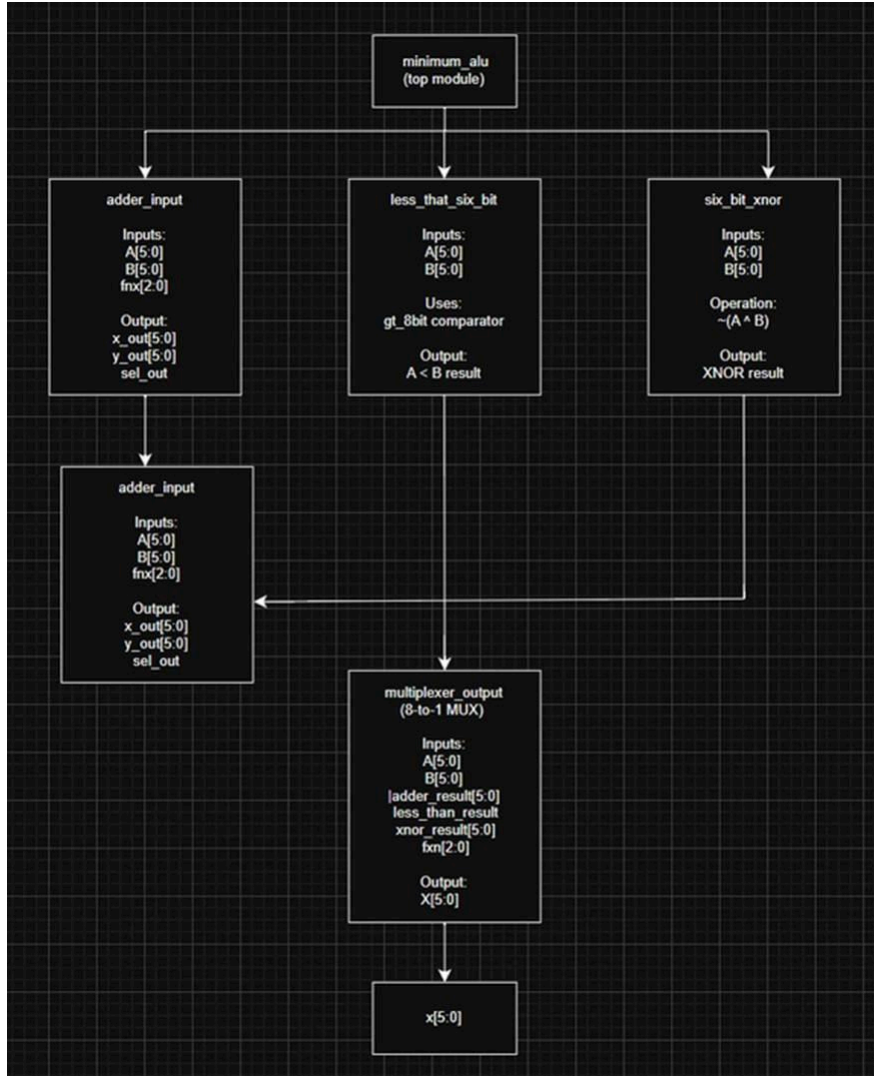


Figure A: Block diagram of `minimum_alu` top-level module

3. Key Module Details

3.1 `adder_input.v`

This module decides what operands and sel signals the ripple adder receives, depending on fcn. For fcn=010 and fcn=011 (negation), I set x_out to zero and put the operand on y_out with sel_out=1, so the adder computes 0 - operand. For addition and subtraction the full A and B are passed through. For the four functions that do not use the adder, the inputs default to A and B with sel=0 and the adder output is simply ignored by the output mux.



fxn	x_out	y_out	sel_out	What the adder computes
010	0	A	1	$0 - A = -A$
011	0	B	1	$0 - B = -B$
110	A	B	0	$A + B$
111	A	B	1	$A - B$
000 / 001 / 100 / 101	A	B	0	Adder output ignored

Table 2: adder_input module routing logic

3.2 less_than_six_bit.v

Getting a signed less-than result without modifying gt_8bit took a bit of thought. The trick I used was to flip the MSB of each 6-bit input before zero-extending them to 8 bits and passing them to gt_8bit. Flipping the MSB converts two's complement into offset binary, where the unsigned ordering of numbers matches the signed ordering. So the unsigned gt_8bit ends up doing the right thing for signed comparison - its output tells me $A \geq B$, and I invert that to get $A < B$.

3.3 six_bit_xnor.v

This module is just a single assigned statement: $\text{result} = \sim(A \wedge B)$. When A and B are identical every bit XNORs to 1, giving 111111 (-1 in two's complement). When every bit differs you get 000000. For anything in between the bits are mixed.

3.4 multiplexer_output.v

This is an 8-to-1 mux implemented using a chain of conditional assignments driven by fxn. For fxn=000 it outputs A, for fxn=001 it outputs B, fxn=010/011/110/111 it outputs the adder result, fxn=100 the less-than result, and fxn=101 the XNOR result. Any undefined fxn value defaults to 6'b000000.

4. Testbench and Testing Strategy

4.1 Approach

I wrote a testbench tb_minimum_alu that instantiates minimum_alu and runs through 21 test vectors. I used \$monitor to print every signal change to the console so I could verify results without needing to inspect the waveform for every single vector. The first test vector (TV1) uses A equal to the last 6 bits of my board number (84 in decimal, 010100 in binary = 20) as required by the assignment.

I tried to cover more than just the typical cases for each function - I included edge cases like negating -32 (the most negative 6-bit value), addition overflow, subtracting from zero, and comparing a negative number against a positive one with fxn=100.



4.2 Test Vectors

TV	fxn	A	B	Expected X	Notes
TV1	000	010100 (20)	010101 (21)	010100 (20)	Pass A - last 6 bits of board number
TV2	000	011100 (28)	111000 (-8)	011100 (28)	Pass A, B is negative
TV3	000	000000 (0)	111111 (-1)	000000 (0)	Pass A = 0, B ignored
TV4	001	101010 (-22)	011011 (27)	011011 (27)	Pass B, A is negative
TV5	001	111111 (-1)	000000 (0)	000000 (0)	Pass B = 0
TV6	010	001100 (12)	000000 (0)	110100 (-12)	Negate A = 12
TV7	010	100000 (-32)	000000 (0)	100000 (-32)	Negate -32 edge case
TV8	011	000000 (0)	001100 (12)	110100 (-12)	Negate B = 12
TV9	011	000000 (0)	100000 (-32)	100000 (-32)	Negate -32 edge case
TV10	100	110000 (-16)	110001 (-15)	000001 (1)	-16 < -15, both negative
TV11	100	001010 (10)	001010 (10)	000000 (0)	Equal inputs, should be false
TV12	100	100001 (-31)	011111 (31)	000001 (1)	Negative < positive
TV13	101	100100 (-28)	100100 (-28)	111111 (-1)	Identical inputs, all XNOR bits = 1
TV14	101	100100 (-28)	011011 (27)	000000 (0)	Inverted inputs, all XNOR bits = 0
TV15	101	110100 (-12)	100110 (-26)	110001	Mixed bit pattern
TV16	110	001000 (8)	000111 (7)	001111 (15)	8 + 7 = 15
TV17	110	011111 (31)	000001 (1)	100000 (-32)	Signed overflow: 31 + 1
TV18	110	111111 (-1)	111111 (-1)	111110 (-2)	-1 + -1 = -2
TV19	111	011111 (31)	011111 (31)	000000 (0)	31 - 31 = 0
TV20	111	000000 (0)	000001 (1)	111111 (-1)	0 - 1 = -1
TV21	111	100001 (-31)	111111 (-1)	100010 (-30)	-31 - (-1) = -30

Table 3: Full test vector set



4.3 Coverage and Shortcomings

All 8 functions are tested with at least two vectors each, and I made sure to include boundary values like 0, +31, and -32. The main gap in my testing is that I did not test exhaustively, there are 4096 possible input combinations for each function and I only tested a handful. I also did not specifically test carry propagation timing in the ripple adder, though since this is a combinatorial design and timing constraints were not defined, this is less of a concern. Another limitation is that I only captured simulation waveforms for fxn=000 and fxn=001; the remaining functions were verified on the board.

5. Simulation Results

5.1 fxn=000 - Pass A (TV1, board number A=20)

This is the first required test vector. A=010100 (20) and fxn=000, so I expect X=010100. The waveform confirms X tracks A immediately with no delay, as expected for a combinatorial design.

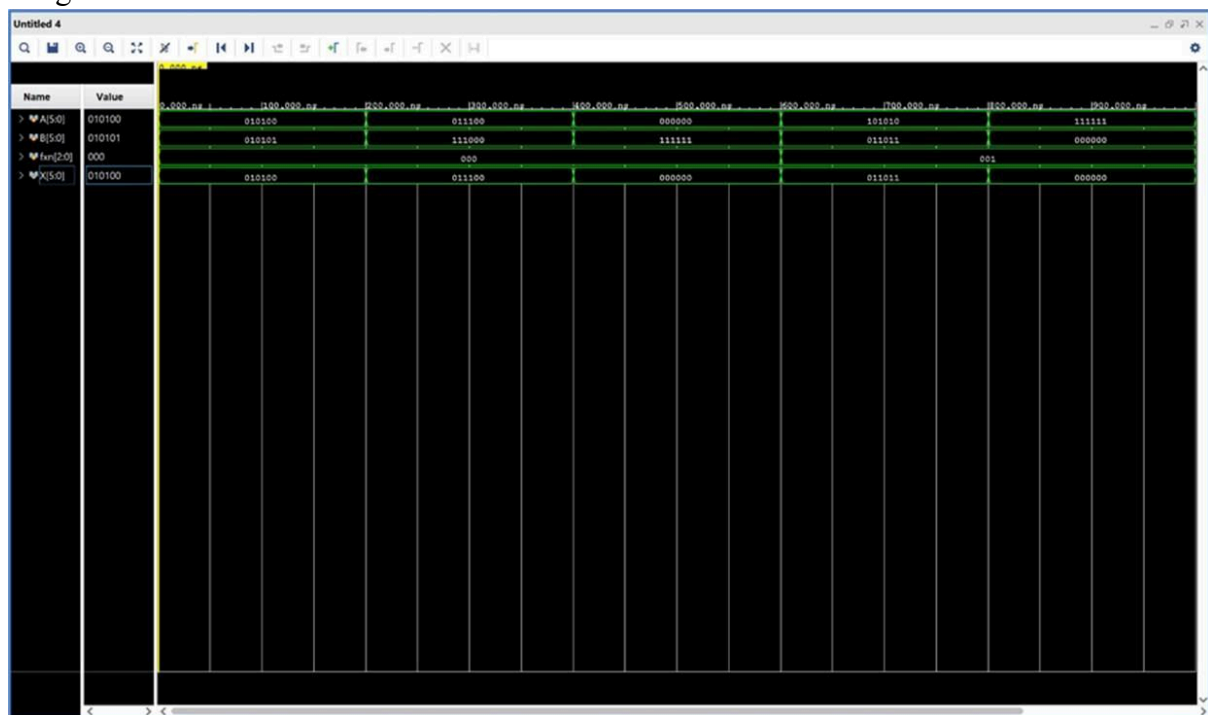


Figure 1: TV1 - fxn=000, A=010100 (20), X=010100 (20)

5.2 fxn=000 - Pass A (TV2, A=28)

A=011100 (28), fxn=000. X=011100 (28) as expected. B=111000 (-8) is a negative value but is correctly ignored.

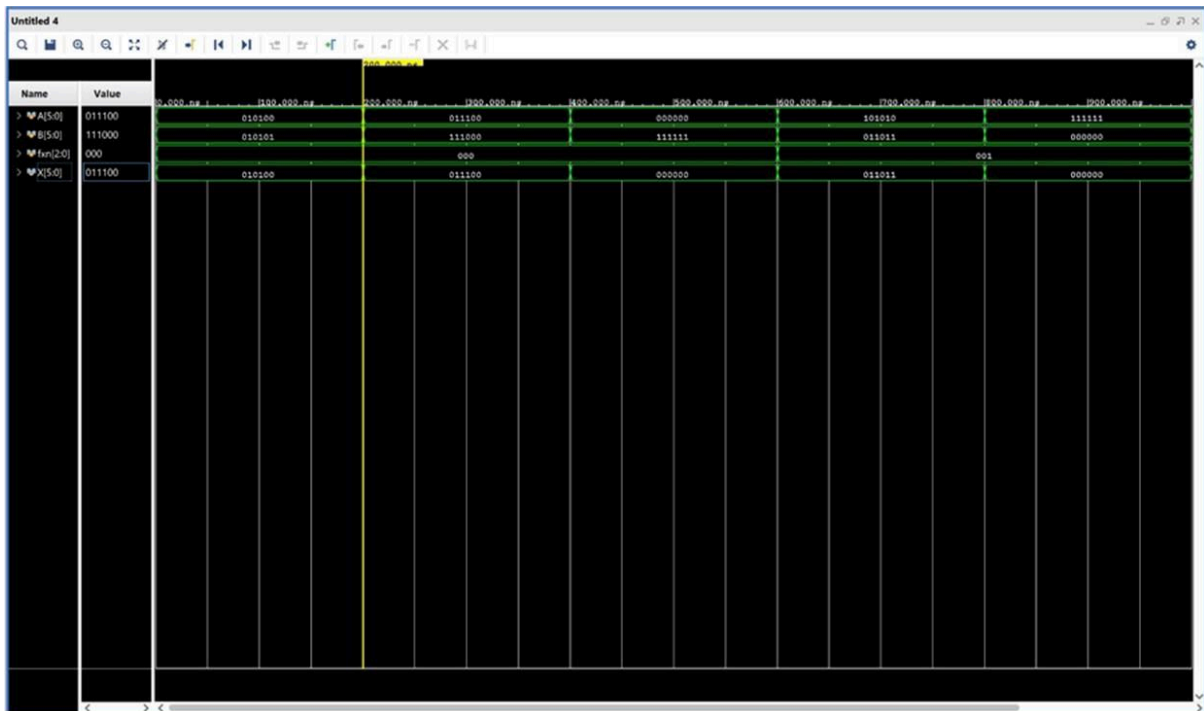


Figure 2: TV2 - fxn=000, A=011100 (28), X=011100 (28)

5.3 fxn=000 - Pass A (TV3, A=0)

A=000000, B=111111 (-1), fxn=000. X=000000, confirming that zero is passed correctly and B has no effect.

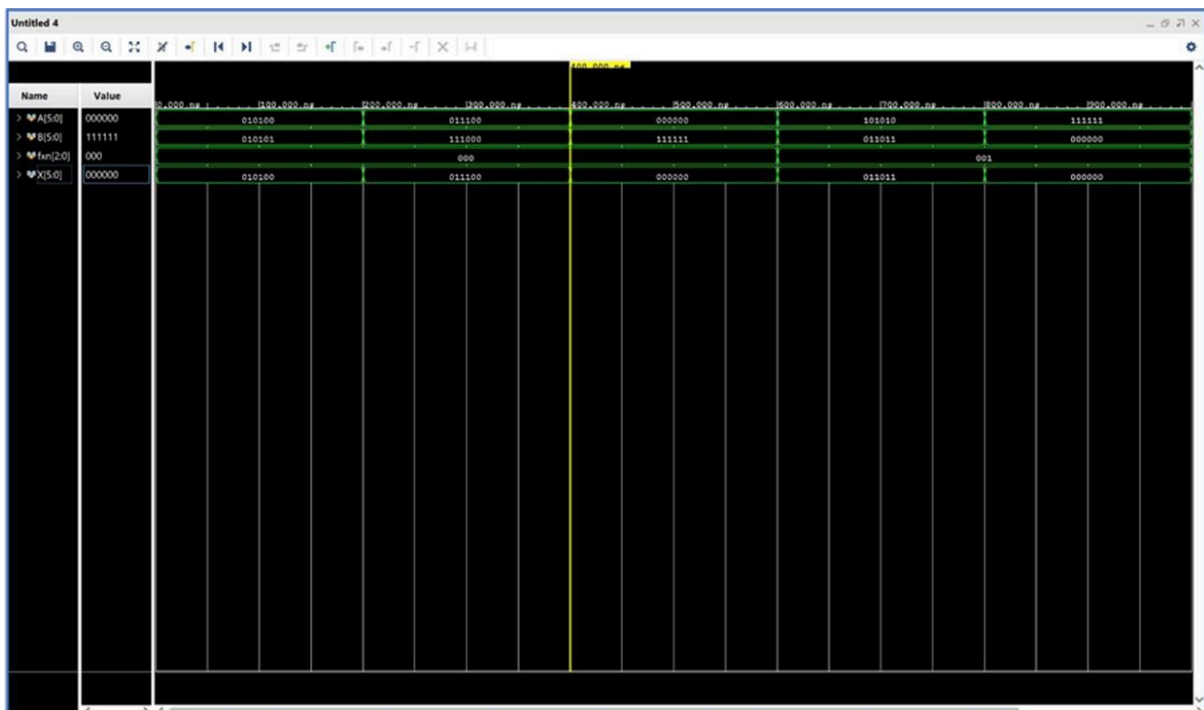


Figure 3: TV3 - fxn=000, A=0, X=0



5.4 fxn=001 - Pass B (TV4, B=27)

fxn changes to 001. A=101010 (-22), B=011011 (27). X=011011 (27) - B is correctly selected and A is ignored.

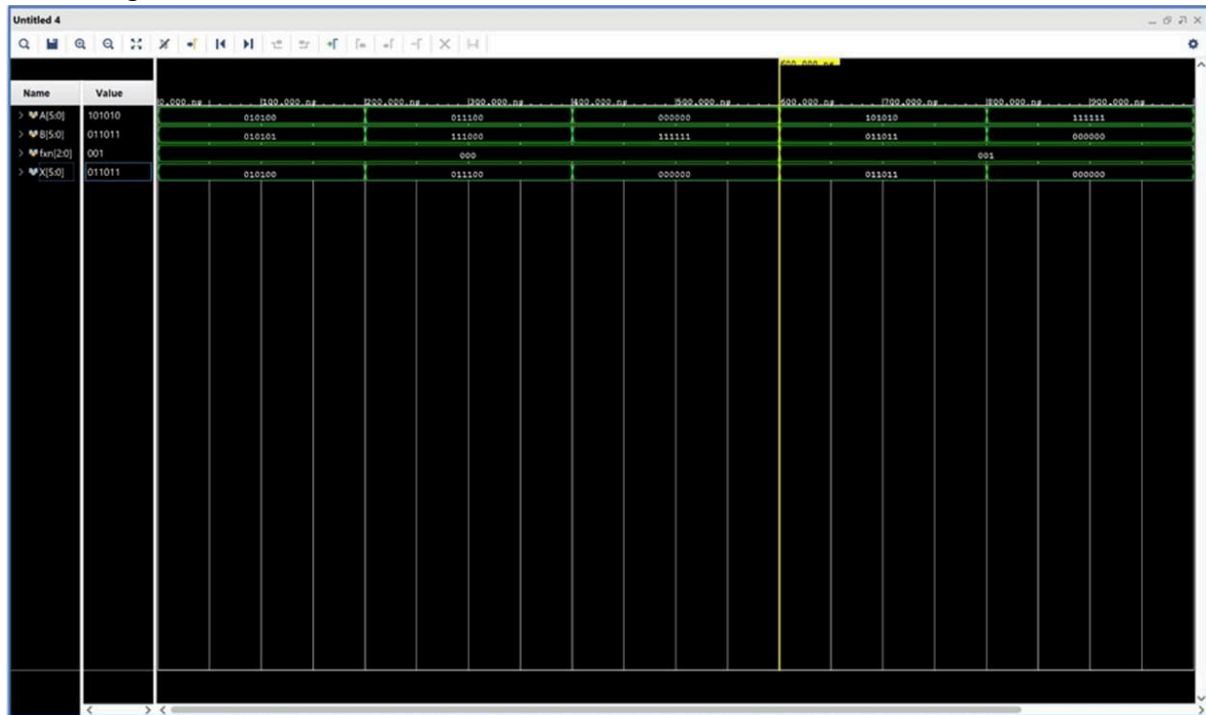


Figure 4: TV4 - fcn=001, B=011011 (27), X=011011 (27)

5.5 fxn=001 - Pass B (TV5, B=0)

A=111111 (-1), B=000000 (0), fcn=001. X=000000. Zero B is handled correctly.

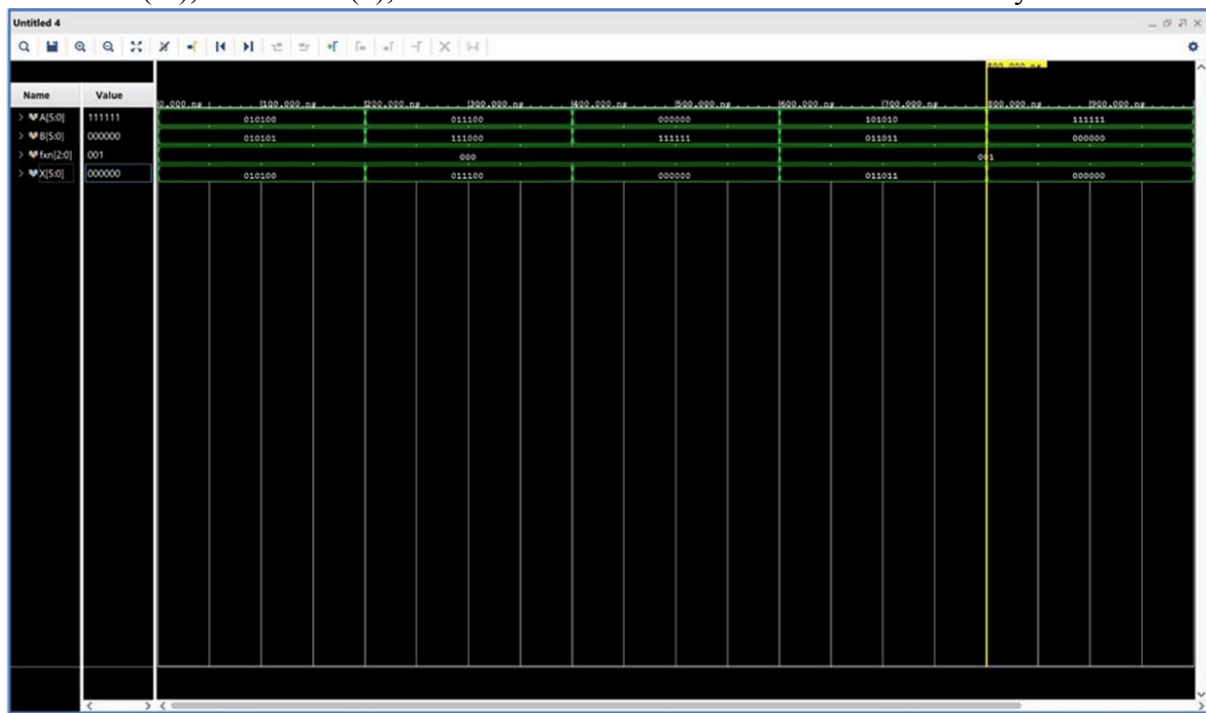


Figure 5: TV5 - fcn=001, B=0, X=0



All remaining test vectors (TV6 through TV21, covering fxn=010 to fxn=111) passed in simulation. Board demonstration results for those functions are shown in Section 7.

6. FPGA Implementation

6.1 Vivado Sources Panel

Figure 6 shows the Sources panel in Vivado after all files were added to the project. The hierarchy is visible with `minimum_alu` at the top, and the testbench `tb_minimum_alu` under Simulation Sources. The constraint file `Basys3_Master.xdc` is shown under Constraints.

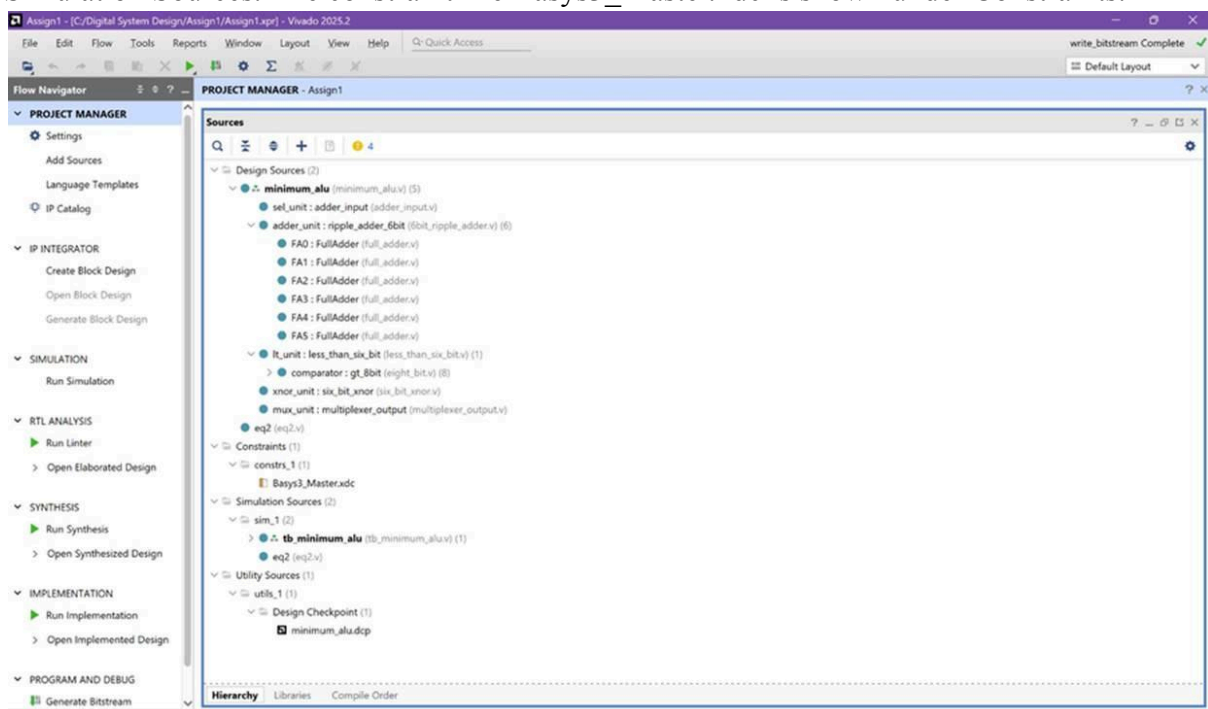


Figure 6: Vivado Sources panel showing the complete design hierarchy

6.2 File Directory on Disk

Figure 7 shows the Windows Explorer view of the source files on disk at `C:\Digital System Design\Assign1\Assign1.srcs\sources_1\new\`. All 8 Verilog source files are present and match the modules listed in the hierarchy.

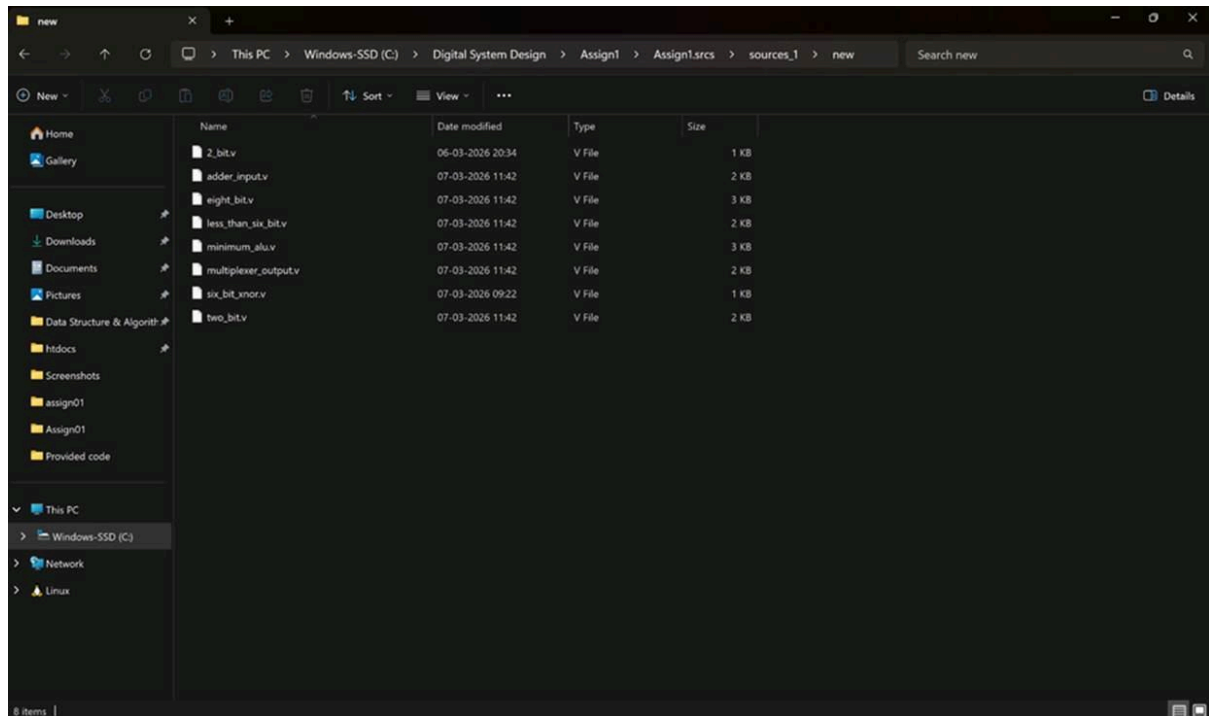


Figure 7: Windows Explorer - 8 Verilog source files at Assign1.srcs\sources_1\new\

6.3 Resource Utilisation

After running synthesis and implementation I checked the utilisation report. The design is very small - only 19 LUTs are used out of the 20,800 available on the Artix-7. The I/O pins take up about 20% of the bonded IOBs, which makes sense given the 15 input and 6 output pins.

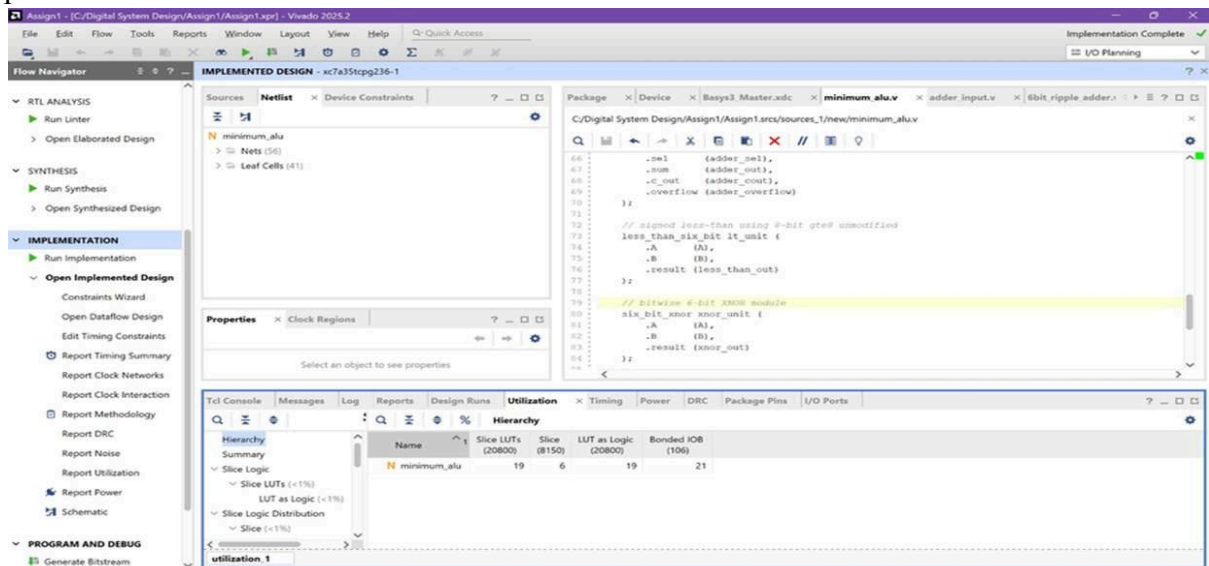


Figure 8: Utilisation hierarchy - minimum_alu: 19 LUTs, 6 Slices, 21 IOBs

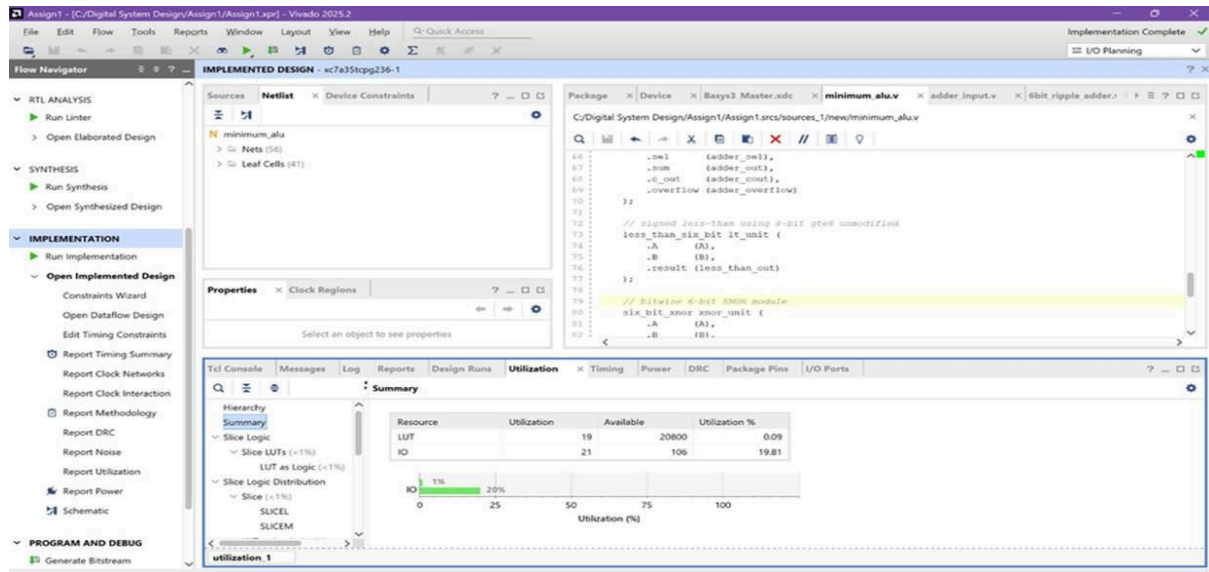


Figure 9: Utilisation summary - LUT: 19/20800 (0.09%), IO: 21/106 (19.81%)

Resource	Used	Available	Utilisation %
LUT	19	20,800	< 0.09%
Slice	6	8,150	< 0.01%
Bonded IOB	21	106	19.81%

Table 4: Resource utilisation summary

6.4 Timing

The timing report showed no failing endpoints. WNS and WHS are both listed as inf, and TNS and THS are both 0.000 ns. This is expected since the design is purely combinatorial and I did not define any clock constraints.

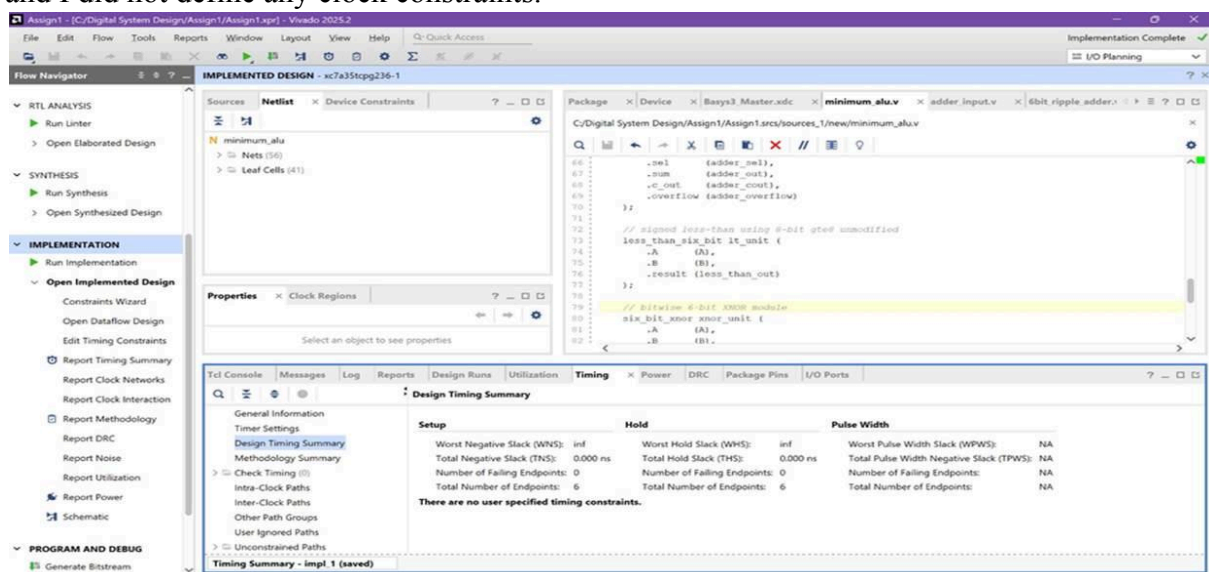


Figure 10: Timing summary - WNS=inf, 0 failing endpoints, no clock constraints



7. Demo

7.1 Switch and LED Mapping

The board inputs and outputs are mapped as follows. All inputs are on the slide switches on the left side of the board and the output appears on the LEDs above them.

Signal	Switches / LEDs	Notes
A[5:0]	SW5 – SW0	SW5 = MSB (bit 5), SW0 = LSB (bit 0)
B[5:0]	SW11 – SW6	SW11 = MSB (bit 5), SW6 = LSB (bit 0)
fxn[2:0]	SW14 – SW12	SW14 = MSB (bit 2), SW12 = LSB (bit 0)
X[5:0] output	LD5 – LD0	LD5 = MSB (bit 5), LD0 = LSB (bit 0)

Table 5: Switch and LED mapping

7.2 Demo Test Cases (Matching the Testbench)

Each test below corresponds directly to a testbench TV. Set switches as shown, then check the LEDs match the expected output. Switch pushed UP = ON (logic 1), DOWN = OFF (logic 0).

Test 1 - fxn=000 (Pass A) [TV1 - Board number 84, last 6 bits]

```
A = 010100 = 20
SW4, SW2 ON | SW5, SW3, SW1, SW0 OFF
B = 010101 = 21
SW10, SW8, SW6 ON | SW11, SW9, SW7 OFF
fxn = 000
SW14, SW13, SW12 OFF

Expected Output:
LD4, LD2 ON -> X = 010100 = 20 [PASS]
```



Figure 11: Test 1 board result



Test 2 - fxn=001 (Pass B) [TV4]

A = 101010 = -22
SW5, SW3, SW1 ON | SW4, SW2, SW0 OFF
B = 011011 = 27
SW10, SW9, SW7, SW6 ON | SW11, SW8 OFF
fxn = 001
SW12 ON | SW14, SW13 OFF

Expected Output:
LD4, LD3, LD1, LD0 ON -> X = 011011 = 27
[PASS]



Figure 12: Test 2 board result

Test 3 - fxn=010 (Negate A) [TV6]

A = 001100 = 12
SW3, SW2 ON | SW5, SW4, SW1, SW0 OFF
B = 000000 = 0
SW6-SW11 all OFF
fxn = 010
SW13 ON | SW14, SW12 OFF

Expected Output:
LD5, LD4, LD2 ON -> X = 110100 = -12 [PASS]



Figure 13: Test 3 board result



Test 4 - fxn=011 (Negate B) [TV8]

```
A = 000000 = 0
  SW0-SW5 all OFF
B = 001100 = 12
  SW9, SW8 ON | SW11, SW10, SW7, SW6 OFF
fxn = 011
  SW13, SW12 ON | SW14 OFF

Expected Output:
  LD5, LD4, LD2 ON -> X = 110100 = -12 [PASS]
```



Figure 14: Test 4 board result

Test 5 - fxn=100 (A < B) [TV10]

```
A = 110000 = -16
  SW5, SW4 ON | SW3, SW2, SW1, SW0 OFF
B = 110001 = -15
  SW11, SW10, SW6 ON | SW9, SW8, SW7 OFF
fxn = 100
  SW14 ON | SW13, SW12 OFF

Expected Output:
  LD0 ON -> X = 000001 = 1 (true: -16 < -15)
[PASS]
```



Figure 15: Test 5 board result



Test 6 - fxn=101 (A XNOR B) [TV13]

A = 100100 = -28
SW5, SW2 ON | SW4, SW3, SW1, SW0 OFF
B = 100100 = -28
SW11, SW8 ON | SW10, SW9, SW7, SW6 OFF
fxn = 101
SW14, SW12 ON | SW13 OFF

Expected Output:
LD5-LD0 all ON -> X = 111111 = -1 [PASS]



Figure 16: Test 6 board result

Test 7 - fxn=110 (A + B) [TV16]

A = 001000 = 8
SW3 ON | SW5, SW4, SW2, SW1, SW0 OFF
B = 000111 = 7
SW8, SW7, SW6 ON | SW11, SW10, SW9 OFF
fxn = 110
SW14, SW13 ON | SW12 OFF

Expected Output:
LD3, LD2, LD1, LD0 ON -> X = 001111 = 15 [PASS]



Figure 17: Test 7 board result



Test 8 - fxn=111 (A - B) [TV19]

```
A = 011111 = 31
  SW4, SW3, SW2, SW1, SW0 ON | SW5 OFF
B = 011111 = 31
  SW10, SW9, SW8, SW7, SW6 ON | SW11 OFF
fxn = 111
  SW14, SW13, SW12 all ON
```

Expected Output:

```
All LEDs OFF -> X = 000000 = 0 [PASS]
```



Figure 18: Test 8 board result

8. Robustness of the Design

I considered a few potential issues with the design:

- **Signed overflow in addition:** When $31 + 1$ is computed, the result wraps to -32 in 6-bit two's complement. This is mathematically correct and expected behaviour - the design does not try to saturate or flag the overflow, it just wraps.
- **Negating -32 :** The most negative 6-bit value is -32 (100000). Negating it gives -32 again because there is no $+32$ in 6-bit two's complement. The design handles this correctly by returning the same bit pattern.
- **Undefined fxn values:** All 8 possible fxn values are explicitly handled. There are no undefined ones in a 3-bit code, so this is not a risk.
- **Switch bounce:** Since this is a purely combinatorial design with no clock, any glitch on the switches goes straight through to the LEDs. In practice this was not visible during the demo, but no debouncing is implemented.
- **Unused adder outputs:** The `c_out` and overflow signals from `ripple_adder_6bit` are not connected at the top level. This was an intentional simplification.



9. Lab E Write-Up

9.1 Development from Lab C

The work in the labs built up incrementally towards this assignment:

- **Lab C:** I designed a single-bit FullAdder using an XOR gate for the sum and a majority function for carry-out. I then chained six of these to make a 6-bit ripple-carry adder. I added subtraction by XOR-inverting the second input y with the sel signal, and feeding sel into carry_in[0], which implements two's complement negation.
- **Lab D:** I created the XDC constraint file to map signals to physical pins on the Basys-3 board - A[5:0] to SW0-SW5, B[5:0] to SW6-SW11, sel to SW12, sum[5:0] to LD0-LD5, c_out to LD6, and overflow to LD7.
- **Lab E:** I ran through the Vivado implementation flow (synthesis, place and route, bitstream generation) for the ripple adder and verified it working on the board by toggling switches and checking the LED outputs against expected results. The four test cases are documented below.
- **Assignment 1:** I integrated the adder as a sub-module inside the Mini-ALU. The adder_input module controls its operands, and the multiplexer_output module selects its output for fxn=010, 011, 110, and 111.

9.2 Lab E Board Test Cases

The following four tests were carried out on the Basys-3 board during Lab E to verify the 6-bit ripple adder/subtractor. Each test shows the switch settings, the expected LED output, and a photo of the board confirming the result.

Lab E - Test 1: Addition $20 + 6 = 26$ [Board number 84]



```
x = 010100 = 20
  SW2, SW4 ON | SW0, SW1, SW3, SW5 OFF
y = 000110 = 6
  SW7, SW8 ON | SW6, SW9, SW10, SW11 OFF
sel = 0 -> SW12 OFF

Expected Output:
LD1, LD3, LD4 ON -> sum = 011010 = 26
LD6 (c_out) OFF | LD7 (overflow) OFF [PASS]
```



Figure 19: Lab E Test 1 - 20+6=26

Lab E - Test 2: Addition 14 + 3 = 17

```
x = 001110 = 14
  SW1, SW2, SW3 ON | SW0, SW4, SW5 OFF
y = 000011 = 3
  SW6, SW7 ON | SW8, SW9, SW10, SW11 OFF
sel = 0 -> SW12 OFF

Expected Output:
LD0, LD4 ON -> sum = 010001 = 17
LD6 (c_out) OFF | LD7 (overflow) OFF [PASS]
```



Figure 20: Lab E Test 2 - 14+3=17



Lab E - Test 3: Subtraction $10 - 3 = 7$

```
x = 001010 = 10
  SW1, SW3 ON | SW0, SW2, SW4, SW5 OFF
y = 000011 = 3
  SW6, SW7 ON | SW8, SW9, SW10, SW11 OFF
sel = 1 -> SW12 ON

Expected Output:
  LD0, LD1, LD2 ON -> sum = 000111 = 7
  LD6 (c_out) ON | LD7 (overflow) OFF [PASS]
```



Figure 21: Lab E Test 3 - $10-3=7$

Lab E - Test 4: Subtraction $3 - 3 = 0$

```
x = 000011 = 3
  SW0, SW1 ON | SW2, SW3, SW4, SW5 OFF
y = 000011 = 3
  SW6, SW7 ON | SW8, SW9, SW10, SW11 OFF
sel = 1 -> SW12 ON

Expected Output:
  All sum LEDs OFF -> sum = 000000 = 0
  LD6 (c_out) ON | LD7 (overflow) OFF [PASS]
```



Figure 22: Lab E Test 4 - $3-3=0$



9.3 Lab E Vivado Screenshots

The screenshots below are from the Lab E Vivado project (LabE2.xpr), showing the ripple_adder_6bit design after implementation. These confirm the adder was successfully synthesised and implemented before being reused in Assignment 1.

Figure 23 shows the Lab E Sources panel. The top-level module is ripple_adder_6bit, with six FullAdder instances (FA0-FA5) beneath it. The constraint file adder.xdc is listed under Constraints.

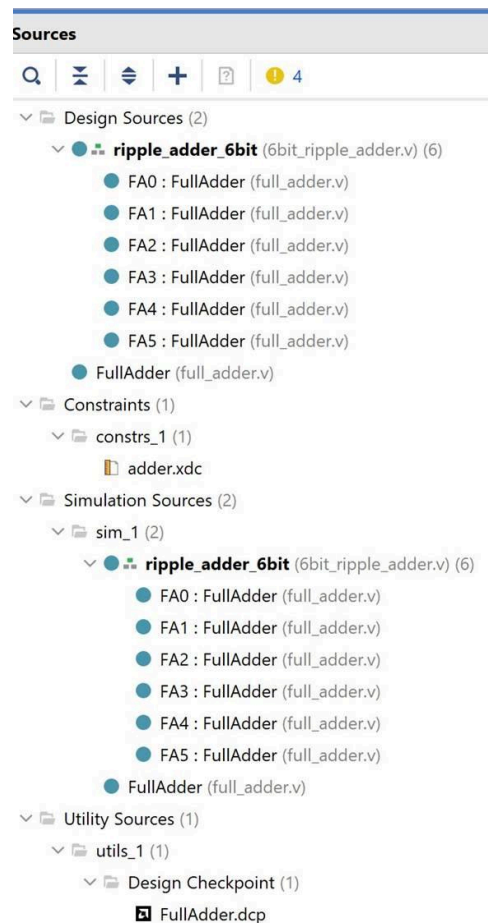


Figure 23: Lab E Vivado Sources panel - ripple_adder_6bit with 6x FullAdder instances

Figure 24 shows the utilisation hierarchy for the Lab E adder. The ripple_adder_6bit uses 9 LUTs and 21 IOBs - fewer than the full ALU as expected, since it is just the adder/subtractor without the mux and comparison logic.

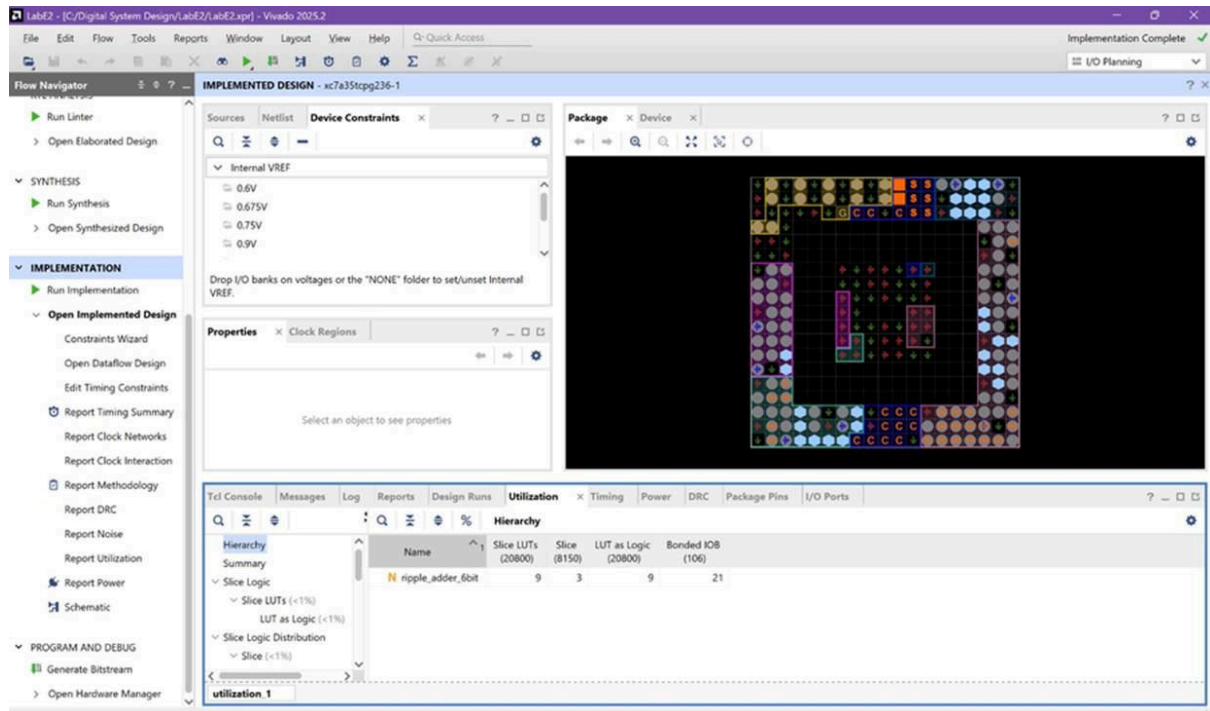


Figure 24: Lab E utilisation hierarchy - ripple_adder_6bit: 9 LUTs, 3 Slices, 21 IOBs

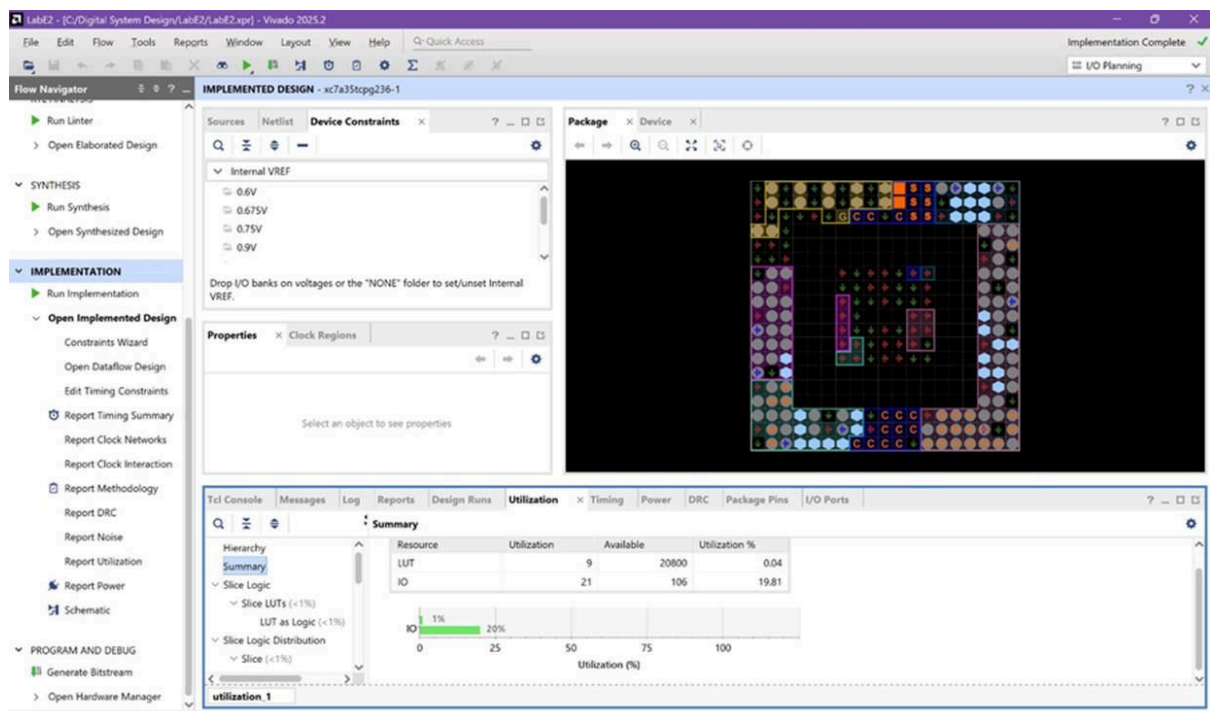


Figure 25: Lab E utilisation summary - LUT: 9/20800 (0.04%), IO: 21/106 (19.81%)

Figure 26 shows the Lab E timing summary. As with the full ALU, WNS=inf and there are 0 failing endpoints, expected for a combinatorial design with no clock constraints.

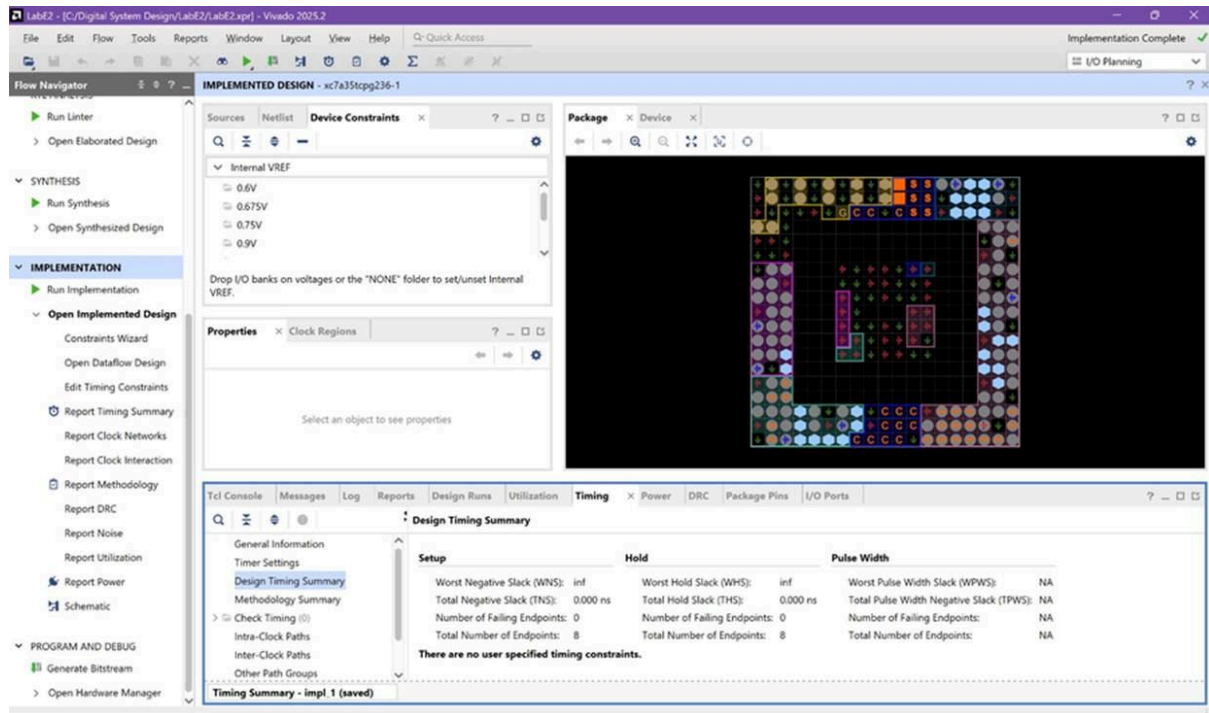


Figure 26: Lab E timing summary - $WNS=inf$, $TNS=0.000ns$, 0 failing endpoints

9.4 Vivado Implementation Flow

The steps I went through to get from Verilog source to a running bitfile on the board were:

- **RTL Analysis / Elaboration:** Vivado reads all the source files, checks for syntax errors, resolves the module hierarchy, and generates an RTL schematic showing the logical structure of the design.
- **Synthesis:** The RTL is translated into a netlist using the Artix-7 primitive library. Vivado maps the logic to LUTs, carry chains, and IOBs, and applies optimisations for area and timing.
- **Implementation:** The synthesised netlist is placed onto actual physical resources on the FPGA fabric. The XDC pin constraints are applied here, binding each signal to a specific package pin. Routing resources are then assigned to connect everything up.
- **Bitstream Generation:** Vivado produces the .bit file that encodes the full FPGA configuration - LUT contents, routing, I/O standards, and all other configuration bits.
- **Programming:** The bitstream is downloaded to the Basys-3 via USB using Vivado's Hardware Manager. The FPGA is configured instantly and the design starts running.

9.5 Notes on Resource Use

I was surprised by how few LUTs the design used - just 9 for the ripple adder alone, and 19 total for the full Mini-ALU. The Artix-7's 6-input LUTs are very efficient at implementing small combinatorial functions, and because this design has no registers at all, everything maps down to just logic. The I/O block usage at around 20% is the most significant constraint, since the design needs 15 input pins and 6 output pins. On the board the LEDs responded immediately to switch changes, which confirmed the combinatorial nature of the design. There were no timing violations reported in either the Lab E or Assignment 1 implementations.



10. Conclusion

Overall I am happy with how the Mini-ALU turned out. All 8 functions work correctly, as verified in simulation with 21 test vectors and then confirmed by running on the board. I managed to reuse all the required lab modules without modifying them, which was one of the key goals of the assignment. The design is clean and modular - each sub-module has a single job, the top level is purely structural, and the resource usage is minimal at under 0.1% of available LUTs. If I were to extend this I would expose the carry-out and overflow outputs to the top-level, add exhaustive simulation coverage, and potentially add pipeline registers to allow operation at a defined clock frequency.